

Welcome to Code Crunch!

C1 | Crunch Convos

WED 2-3PM Weekly | Spring 2025

Attendance

Earn a Certificate of
Completion by attending
at least 70% of the
workshops



linktr.ee/CODE.CRUNCH



Unit #3

C1 | Two Pointers & Sliding Window



Two Pointer Technique

A strategy that uses two pointers to iterate through a data structure, typically arrays or linked lists, to solve problems efficiently.

Use Cases:

- Reversing a list
- Detecting cycles in linked lists
- Merging sorted arrays

Benefits:

- Reduces time complexity
- Often uses constant extra space



Types of Two Pointer Approaches

1. Opposite Directions:

- One pointer starts at the beginning, and the other starts at the end.
- They move toward each other based on a condition

2. Same Direction:

- Both pointers start at different or same positions and move in the same direction (often at different speeds).
- One pointer may "chase" the other, helping to maintain a window or segment.

Reversing a List

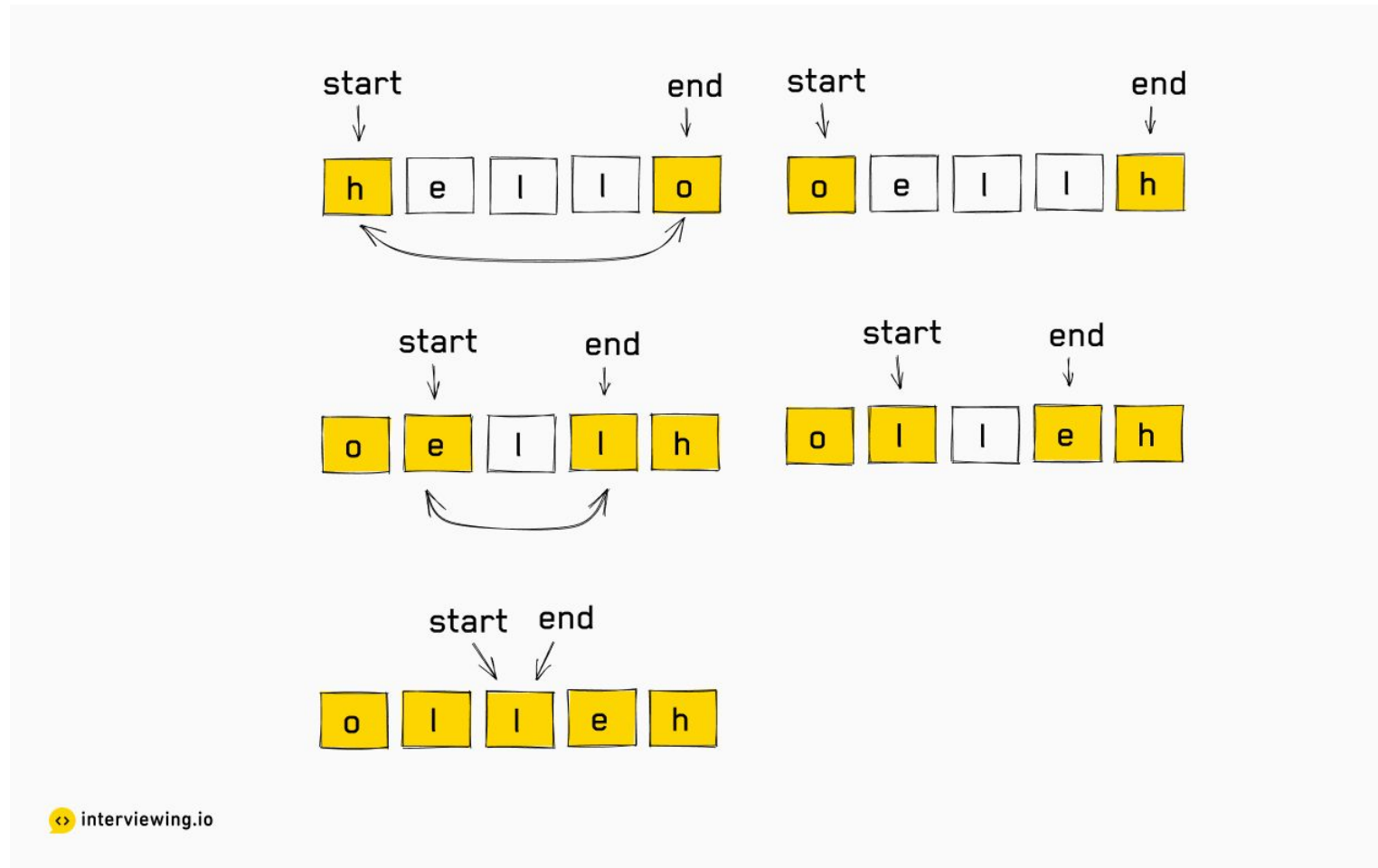


Fig. : Wagner, Tom . "How to Reverse a String (With Solutions in Python, Java & JavaScript)",
<https://interviewing.io/questions/reverse-string>

Reversing a List (Cont.)

```
def reverse_list(nums):  
    # Initialize two pointers  
    left = 0  
    right = len(nums) - 1  
    while left < right:  
        # Swap elements  
        temp = nums[left]  
        nums[left] = nums[right]  
        nums[right] = temp  
  
        left += 1    # Move left pointer forward  
        right -= 1  # Move right pointer backward
```



Problem

A **palindrome** is a sequence that reads the same backward as forward. Write a function that takes string `str` as a parameter and outputs either `True` or `False` if it is a palindrome.

Example 1

Input: `str = racecar`

Output: `True`

Example 2

Input: `str = star`

Output: `False`



Sliding Window

A method for solving problems that involve finding subarrays or substrings that satisfy certain conditions by maintaining a "window" that moves over the data structure.

Use Cases:

- Finding the maximum sum subarray of size k
- Longest substring without repeating characters

Benefits:

- Achieves linear time complexity
- Efficiently manages data within a range

Sliding Window (Cont.)

- Define a window using two pointers that represents a subset of the data.
- Expand or shrink the window based on specific criteria to find the optimal solution.

Types of Sliding Windows:

- Fixed Size: The window size remains constant while sliding.
- Dynamic Size: The window size can grow or shrink based on the problem's requirements.



Problem

Given an unsorted list of integers `lst`, calculate the maximum sum of `k` consecutive elements.

Example 1

Input: `lst = [1, 3, 5, 2, 8, 1, 9]`, `k = 3`

Output: 18



Bonus Problem

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

- The length of the subarray is `k`, and
- All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

Example 1

Input: `nums = [1,5,4,2,9,9,9]`, `k = 3`

Output: 15



Attendance



linktr.ee/CODE.CRUNCH



Join us for the next sessions



Crunch Convos: WED 2PM - 3PM

Crunch Code: FRI 1PM - 2PM



RSVP lu.ma/CODECRUNCH



Your feedback helps us improve. Share your thoughts!

Go to our website: go.fiu.edu/codecrunch